DevOps Technical Assessment - Completion Report

Docker VMs, Ansible Automation, and Hardened Nginx Reverse Proxy

Final Implementation Record

1. Project Summary

This document records the complete implementation of the DevOps Technical Assessment: a three-container Docker "VM" infrastructure, configured end-to-end with Ansible, serving dynamic web content behind a hardened Nginx reverse proxy with TLS, load balancing, and automatic failover.

Environment used: Windows 11, WSL2 (Ubuntu 24.04.1 LTS), Docker Desktop with WSL2 integration, Windows Terminal.

Repository: `https://github.com/laxmikumb/devops-docker-ansible-assignment` (public)

All work was completed at zero cost, using only free and open-source tools, per the constraint in Section 5 of the brief.

Environment setup commands:

```
sudo apt update
sudo apt install -y ansible openssl curl git nginx
docker --version
ansible --version
git --version

mkdir -p ~/devops-assignment/docker ~/devops-assignment/ansible \
  ~/devops-assignment/nginx ~/devops-assignment/docs
cd ~/devops-assignment
git init
```

---

2. What Was Built, Section by Section

2.1 Docker Containers (Section 4.1)

- Built a custom Ubuntu 22.04-based image (`assignment-vm`) with OpenSSH, Nginx, UFW, and supporting tools pre-installed.
- Created a custom Docker bridge network `vmnet` with a defined subnet `172.28.0.0/24` (not Docker's default auto-assigned network).
- Launched three containers — `vm1`, `vm2`, `vm3` — each with `--restart unless-stopped` and `--cap-add=NET_ADMIN` (required for UFW to manage firewall rules inside the container).
- An `entrypoint.sh` script configures UFW with a default-deny inbound policy on container start, opening SSH only from the `vmnet` subnet, and opening ports 80/443 only on vm1 (the designated proxy node via a `ROLE=proxy` environment variable).

Commands used:

```
docker network create --subnet=172.28.0.0/24 vmnet
```

```
        docker build -t assignment-vm .

        docker run -d --name vm1 --hostname vm1 --network vmnet --ip
172.28.0.11 \
          -p 127.0.0.1:2201:22 -p 127.0.0.1:8080:80 -p 127.0.0.1:8443:443 \
          --cap-add=NET_ADMIN --restart unless-stopped -e ROLE=proxy
assignment-vm

        docker run -d --name vm2 --hostname vm2 --network vmnet --ip
172.28.0.12 \
          -p 127.0.0.1:2202:22 --cap-add=NET_ADMIN --restart unless-stopped
\
          -e ROLE=backend assignment-vm

        docker run -d --name vm3 --hostname vm3 --network vmnet --ip
172.28.0.13 \
          -p 127.0.0.1:2203:22 --cap-add=NET_ADMIN --restart unless-stopped
\
          -e ROLE=backend assignment-vm
```

Verification (auto-restart on crash):

```
        docker ps
        docker exec vm2 pkill -9 nginx
        sleep 3
        docker ps
        docker inspect --format='{{.State.StartedAt}}' vm2
```

2.2 Hardened SSH (Section 4.2)

- A dedicated `deployer` user was created in the image, authenticating via an ED25519 SSH key pair generated with `ssh-keygen`.
- `sshd_config` was modified at build time to set `PasswordAuthentication no` and `PermitRootLogin no`.

Commands used:

```
        ssh-keygen -t ed25519 -f docker/vm_key -N ""
        cp docker/vm_key.pub docker/authorized_keys
```

Verification:

```
        ssh -i docker/vm_key -p 2201 -o StrictHostKeyChecking=no
deployer@localhost "hostname"
        ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
-p 2201 deployer@localhost
        ssh -i docker/vm_key -p 2201 root@localhost
```

2.3 Local Domains and TLS (Section 4.3)

- `public.vm1.local`, `public.vm2.local`, and `public.vm3.local` were mapped in the Windows hosts file (`C:\Windows\System32\drivers\etc\hosts`) to `127.0.0.1`.
- A self-signed TLS certificate (365-day validity, with a SAN extension) was generated via OpenSSL for `public.vm1.local`.
- Nginx on vm1 was configured to redirect all HTTP traffic to HTTPS with a 301 redirect.

Commands used:

```
        mkdir -p nginx/certs
        openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
          -keyout nginx/certs/vm1.key -out nginx/certs/vm1.crt \
          -subj /CN=public.vm1.local -addext
subjectAltName=DNS:public.vm1.local

        docker cp nginx/certs/vm1.crt vm1:/etc/ssl/certs/vm1.crt
```

```
docker cp nginx/certs/vm1.key vm1:/etc/ssl/private/vm1.key
```

Verification:

```
ping -c 2 public.vm1.local
curl -Ik http://public.vm1.local:8080
curl -Ik https://public.vm1.local:8443
```

2.4 Ansible Inventory (Section 4.4)

- Ansible was installed on the host (WSL2 Ubuntu).
- `inventory.ini` lists all three VMs; `ansible.cfg` sets the SSH private key path and vault password file so no credentials are passed by hand on each run.

inventory.ini:

```
[webservers]
vm1 ansible_host=127.0.0.1 ansible_port=2201
vm2 ansible_host=127.0.0.1 ansible_port=2202
vm3 ansible_host=127.0.0.1 ansible_port=2203

[webservers:vars]
ansible_user=deployer
ansible_python_interpreter=/usr/bin/python3
```

ansible.cfg:

```
[defaults]
inventory = inventory.ini
private_key_file = ../docker/vm_key
host_key_checking = False
retry_files_enabled = False
vault_password_file = .vault_pass.txt
```

Verification:

```
cd ansible
ansible all -m ping
```

2.5 Ansible Playbook and Roles (Section 4.5)

- The playbook (`site.yml`) is structured into three roles: `common`, `ssh`, and `nginx` — not a single flat file.
- `common` creates an `ansible_user` with a vault-encrypted password and grants passwordless sudo via a dedicated file in `/etc/sudoers.d/`, then installs `curl`, `git`, and `docker.io`.
- `ssh` enforces the hardened SSH settings idempotently.
- `nginx` installs Nginx and deploys the dynamic page and site configuration.
- All secrets are stored using `ansible-vault encrypt_string` — no plaintext passwords are committed anywhere in the repository.

Commands used to set up the vault:

```
echo "MySecretVaultPass123" > .vault_pass.txt
ansible-vault encrypt_string 'MyStrongPassword123!' \
  --name 'vault_ansible_user_password' > group_vars/all/vault.yml
```

roles/common/tasks/main.yml (key task):

```
- name: Create ansible_user
  ansible.builtin.user:
    name: ansible_user
    password: "{{ vault_ansible_user_password | string |
password_hash('sha512', 'fixedsalt123') }}"
```

```
            shell: /bin/bash
            state: present
```

Running and verifying idempotency:

```
        ansible-playbook site.yml | tee ../docs/run1.log
        ansible-playbook site.yml | tee ../docs/run2.log
```

Second run's PLAY RECAP showed `changed=0` across all three hosts. Both logs saved in `docs/run1.log` and `docs/run2.log`.

2.6 Nginx Dynamic Pages (Section 4.6)

- Nginx was installed on all three VMs via Ansible.
- Each VM serves a page using Nginx's built-in SSI (Server Side Includes) to display its hostname and the current server time, generated fresh on every request — no backend application code or cron job required.

roles/nginx/templates/index.shtml.j2:

```
        <html>
        <head><title>{{ inventory_hostname }}</title></head>
        <body>
          <h1>Host: <!--#echo var="hostname" --></h1>
          <p>Server time: <!--#echo var="date_local" --></p>
        </body>
        </html>
```

Verification (dynamic content proof):

```
        docker exec vm2 curl -s http://localhost
        sleep 2
        docker exec vm2 curl -s http://localhost
```

2.7 Reverse Proxy, Load Balancing, and Failover (Section 4.7)

- Nginx on vm1 was configured as a reverse proxy with three routes:
  - `/vm2/` and `/vm3/` route directly to their respective backend VMs.
  - `/app/` load-balances across vm2 and vm3 using an `upstream` block with passive health checking (`max_fails`, `fail_timeout`).

Key vm1 Nginx config (deployed via Ansible template, conditional on `inventory_hostname == 'vm1'`):

```
upstream app_backend {
    server vm2:80 max_fails=2 fail_timeout=5s;
    server vm3:80 max_fails=2 fail_timeout=5s;
}

server {
    listen 80;
    server_name public.vm1.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name public.vm1.local;
    ssl_certificate     /etc/nginx/ssl/selfsigned.crt;
    ssl_certificate_key /etc/nginx/ssl/selfsigned.key;

    location / { ssi on; root /var/www/html; index index.shtml; }
    location /vm2/ { proxy_pass http://vm2/; proxy_set_header Host
$host; }
    location /vm3/ { proxy_pass http://vm3/; proxy_set_header Host
```

```
$host; }
    location /app/ {
        proxy_pass http://app_backend/;
        proxy_next_upstream error timeout http_502 http_503
http_504;
        proxy_connect_timeout 2s;
    }
}
```

Verification (routing + load balancing):

```
        curl -k https://public.vm1.local:8443/vm2/
        curl -k https://public.vm1.local:8443/vm3/
        curl -k https://public.vm1.local:8443/app/
        curl -k https://public.vm1.local:8443/app/
```

Verification (live failover test):

```
        ( while true; do curl -sk -o /dev/null -w "%{http_code} - %
{time_total}s\n" \
          https://public.vm1.local:8443/app/; sleep 1; done ) >
docs/failover.log 2>&1 &

        sleep 5
        tail -5 docs/failover.log

        docker stop vm2
        sleep 10
        tail -10 docs/failover.log

        docker start vm2
        sleep 10
        tail -15 docs/failover.log

        kill %1
        for i in 1 2 3 4 5 6; do curl -sk https://public.vm1.local:8443/app/
| grep Host; done
```

`vm2` was stopped — all requests continued succeeding (HTTP 200) via vm3, with only a single brief latency spike (~2 seconds) on the request that coincided with the stop, and zero failed requests. `vm2` was then restarted and automatically rejoined rotation (confirmed by alternating `vm2`/`vm3` responses) with **no manual Nginx reload** performed at any point. The full log is saved in `docs/failover.log`.

---

3. Errors Encountered and How They Were Resolved

This section documents real issues hit during the build, since troubleshooting infrastructure problems is itself part of the assessed skill set.

3.1 Docker build failing on package downloads (`400 Bad Request`)

Symptom: `docker build` repeatedly failed partway through `apt-get install`, each time on a different package, with errors like `Failed to fetch ... 400 Bad Request`.

Root cause: a combination of factors on the Windows/WSL2 network path was intermittently corrupting larger HTTP downloads — confirmed to involve the local ISP/DNS resolution path, since the exact same file (`libcbor0.8`) failed consistently across three different Ubuntu mirrors.

Resolution: 1. Added `-o Acquire::Retries=5` and `--fix-missing` to the `apt-get` commands in the Dockerfile for resilience. 2. Switched WSL2's DNS resolution to Google/Cloudflare, which resolved the issue permanently:

```
        sudo tee /etc/resolv.conf <<'EOF'
        nameserver 8.8.8.8
        nameserver 1.1.1.1
        EOF
        docker build --no-cache -t assignment-vm .
```

3. Along the way, also identified and temporarily disabled a Quick Heal firewall component that was a contributing factor on this machine.

3.2 `docker kill` not triggering the `--restart unless-stopped` policy

Symptom: using `docker kill vm2` to simulate a crash did not cause the container to restart, even with the restart policy correctly configured.

Root cause: Docker treats `docker kill`/`docker stop` as an *intentional* user action. The `unless-stopped` policy is specifically designed to NOT restart a container the user deliberately stopped — this is correct, expected Docker behavior, not a bug.

Resolution: simulated a genuine, unplanned crash instead by killing the Nginx process *from inside* the running container, which caused the entrypoint script's `wait` to exit naturally — Docker then correctly auto-restarted the container, confirmed via a changed `StartedAt` timestamp.

```
        docker exec vm2 pkill -9 nginx
        sleep 3
        docker inspect --format='{{.State.StartedAt}}' vm2
        docker ps
```

3.3 WSL2 host unable to reach custom Docker network IPs directly

Symptom: after the first successful container run, SSH and ping to container IPs (e.g. `172.28.0.11`) began timing out, and `ip route` on the WSL2 host showed no route to the `172.28.0.0/24` subnet at all.

Root cause: a known limitation/quirk of Docker Desktop's network bridge on Windows/WSL2 — the custom bridge network's routes are not always exposed to the WSL2 host terminal, particularly after a Docker Desktop or Windows restart.

Resolution: rather than relying on direct container-IP access from the host, each container's SSH (and vm1's web) ports were published to `127.0.0.1` specifically, keeping them inaccessible from the wider network while making them reachable from the host via `localhost`. Container-to-container traffic (vm1's reverse proxy reaching vm2/vm3) was unaffected, since that traffic stays inside the Docker network itself and does not depend on the WSL2 host's routing table. This is documented as a key assumption in the project README.

```
        docker stop vm1 vm2 vm3
        docker rm vm1 vm2 vm3
        docker run -d --name vm1 --hostname vm1 --network vmnet --ip
172.28.0.11 \
          -p 127.0.0.1:2201:22 -p 127.0.0.1:8080:80 -p 127.0.0.1:8443:443 \
          --cap-add=NET_ADMIN --restart unless-stopped -e ROLE=proxy
assignment-vm
        # (vm2, vm3 recreated the same way with their own SSH ports)
```

```
ssh -i docker/vm_key -p 2201 deployer@localhost "hostname"
```

3.4 Ansible `password_hash` filter failing on vault-encrypted values

Symptom: `ansible-playbook site.yml` failed with `secret must be unicode or bytes, not AnsibleVaultEncryptedUnicode`.

Root cause: a known compatibility issue between newer Ansible Core versions and the `password_hash` Jinja2 filter when applied directly to a vault-encrypted variable.

Resolution: added an explicit `| string` filter before `| password_hash(...)` in the `common` role, forcing the decrypted vault value to a plain string first.

```
password: "{{ vault_ansible_user_password | string | password_hash('sha512') }}"
```

3.5 Nginx reload handler failing (`Service is in unknown state`)

Symptom: the `nginx` role's tasks all succeeded, but the handler that reloads Nginx failed with `Service is in unknown state`.

Root cause: these containers have no `systemd` or other real init system, so Ansible's generic `service` module cannot query Nginx's status the way it would on a full VM.

Resolution: changed both the `nginx` and `ssh` role handlers to run the reload/restart commands directly rather than going through the `service` module's status-checking logic.

```
# roles/nginx/handlers/main.yml
- name: reload nginx
  ansible.builtin.command: nginx -s reload

# roles/ssh/handlers/main.yml
- name: restart sshd
  ansible.builtin.command: service ssh restart
```

3.6 Idempotency broken by random password-hash salting

Symptom: after fixing 3.4, the playbook ran successfully, but every repeated run still reported `changed=1` on the `Create ansible_user` task, breaking the idempotency requirement.

Root cause: Ansible's `password_hash` filter generates a new random cryptographic salt on every run by default, so the resulting hash looks "different" each time even when the underlying password is unchanged.

Resolution: passed an explicit fixed salt as a second argument, making the hash deterministic across runs.

```
password: "{{ vault_ansible_user_password | string | password_hash('sha512', 'fixedsalt123') }}"
```

```
ansible-playbook site.yml | tee ../docs/run1.log
ansible-playbook site.yml | tee ../docs/run2.log
```

Second run correctly showed `changed=0` on all hosts.

3.7 Large heredoc pastes truncating in the terminal

Symptom: very long multi-line `cat > file << EOF ... EOF` blocks (e.g. the full README) silently failed to create the target file, with no visible error.

Root cause: pasting very large blocks of text into the terminal in one go occasionally dropped or interrupted part of the paste.

Resolution: split large file creation into smaller sequential chunks using append mode, building the file up incrementally and verifying its growing size after each chunk.

```
cat > docs/README.md << 'EOF'
# DevOps Technical Assessment - Implementation Documentation
## 1. Docker Setup
...
EOF
ls -la docs/README.md

cat >> docs/README.md << 'EOF'

## 4. Ansible Automation
...
EOF
ls -la docs/README.md
# repeated in further chunks until the full file was built
```

3.8 Nginx role silently overwriting vm1's redirect/proxy config

Symptom: after a later `ansible-playbook` run, `curl -Ik http://public.vm1.local:8080` returned `200 OK` instead of the expected `301` redirect, even though the HTTPS side and routing still appeared to work.

Root cause: the `nginx` role's site-config template (`default.conf.j2`) applied the same generic configuration to all three hosts, including vm1. This generic config served content directly on port 80 with no redirect, and did not carry vm1's TLS-redirect-and-reverse-proxy setup or the `/app/` upstream's passive health-check parameters (`max_fails`/`fail_timeout`).

Resolution: rewrote the template using a Jinja2 conditional so vm1 receives its full TLS + redirect + reverse-proxy + load-balanced configuration, while vm2 and vm3 continue to receive the simple SSI-only page config.

```
{% if inventory_hostname == 'vm1' %}
upstream app_backend {
    server vm2:80 max_fails=2 fail_timeout=5s;
    server vm3:80 max_fails=2 fail_timeout=5s;
}
server {
    listen 80;
    server_name public.vm1.local;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    ...
}
{% else %}
server {
    listen 80 default_server;
```

```
    server_name _;
    ssi on;
    root /var/www/html;
    index index.shtml;
}
{% endif %}
```

```
        ansible-playbook site.yml
        curl -Ik http://public.vm1.local:8080
        curl -Ik https://public.vm1.local:8443
        ansible-playbook site.yml | tee ../docs/run1.log
        ansible-playbook site.yml | tee ../docs/run2.log
```

Re-confirmed `changed=0` on the second run, and re-verified the 301 redirect, HTTPS response, fixed-path routing, and load balancing all independently.

3.9 Stray private key committed to the public repository

Symptom: an unrelated file, `privkey.pem` (a genuine OpenSSH private key, confirmed via the `file` command), was found sitting in the project root and had already been pushed in the very first commit to the public GitHub repository.

Root cause: unclear — the file's origin could not be definitively traced; it may have been a leftover artifact from an earlier manual testing step. Regardless of origin, a private key must never be present in a public repository.

Resolution: the file was deleted, `*.pem` and the filename itself were added to `.gitignore`, and since the key had already been committed, the repository's entire history was rewritten: a new orphan branch was created containing only a single clean commit of the current, correct file set, and the old `main` branch was force-replaced on GitHub with this clean branch.

```
        rm privkey.pem
        echo "privkey.pem" >> .gitignore
        echo "*.pem" >> .gitignore

        git checkout --orphan clean-main
        git add -A
        git commit -m "Clean history: remove exposed private key, fix vm1
Nginx redirect template"

        git branch -D main
        git branch -m main
        git push origin main --force
```

The repository's public history now contains no trace of the key at any point.

3.10 One anomalous 33-second request during an early failover test

Symptom: in an earlier run of the failover test, one request in the logged curl loop took approximately 33 seconds to complete (still returning `200`, but far outside the normal sub-second response time).

Root cause: this occurred during the same session where the Nginx template bug (3.8) was still active and vm2 was being restarted; the unusually long delay is consistent with Nginx briefly attempting to reach a backend in an inconsistent state while the config was in flux.

Resolution: after fixing the template issue in 3.8, the entire failover test was re-run cleanly from scratch. The repeat test showed only the expected brief (~2 second) latency spikes at the exact moments vm2 was stopped and restarted, with every request still returning `200`, and no anomalous multi-second delays. This clean log is the one included in the final submission.

---

4. Final Verification Summary

Requirement	Verified Via	Result
Containers running, custom subnet	`docker ps`, `docker network inspect vmnet`	3 containers on `172.28.0.0/24`
Auto-restart on crash	`docker exec vm2 pkill -9 nginx` + `docker inspect`	Confirmed, new StartedAt timestamp
Firewall default-deny	UFW rules in `entrypoint.sh`	SSH only from vmnet; 80/443 only on vm1
Key-only SSH	`ssh -i vm_key -p 2201 deployer@localhost`	Success, no password prompt
Password login rejected	`ssh -o PreferredAuthentications=password ...`	`Permission denied (publickey)`
Root login rejected	`ssh -i vm_key root@localhost`	`Permission denied (publickey)`
Domain resolution	`ping public.vm1.local`	Resolves to 127.0.0.1
HTTPS + redirect	`curl -Ik http://...:8080` / `https://...:8443`	301 redirect, then 200 OK
Ansible connectivity	`ansible all -m ping`	3x `pong`
Idempotency	`ansible-playbook site.yml` x2	Run 2: `changed=0` on all hosts
Dynamic pages	repeated `curl` to each VM	Timestamp changes every request
Fixed-path routing	`curl .../vm2/`, `curl .../vm3/`	Correct VM returned each time
Load balancing	repeated `curl .../app/`	Alternates vm2/vm3
Failover	background curl loop + `docker stop/start vm2`	Zero failed requests; auto-rejoin, no manual reload

---

5. Repository and Submission

- **GitHub repository (public):** `https://github.com/laxmikumb/devops-docker-ansible-assignment`
- Contains: Docker configuration files, Ansible inventory/roles/vaulted secrets, Nginx configuration, and full project documentation (`docs/README.md`), plus evidence logs (`run1.log`, `run2.log`, `failover.log`).

- `.gitignore` excludes the private SSH key (`docker/vm_key`), the vault password file (`.vault_pass.txt`), and the TLS private key (`nginx/certs/vm1.key`) — no sensitive material was committed.

Remaining submission steps: 1. Record the demonstration video covering all items in Section 7.2 of the brief (full command sequence below). 2. Upload the video to Google Drive with sharing set to "Anyone with the link can view." 3. Submit the GitHub repository link and the video link via the submission form: `https://forms.gle/SfX9aA8qUDXZgHs6A`

---

6. Complete Demonstration Video Command Sequence

The commands below, run in order on camera, cover every bullet point in Section 7.2 of the brief.

1. Containers running + auto-restart on crash

```
        docker ps
        docker exec vm2 pkill -9 nginx
        sleep 3
        docker ps
```

2. SSH: key login works, password and root login rejected

```
        cd /devops-assignment
        ssh -i docker/vm_key -p 2201 -o StrictHostKeyChecking=no
deployer@localhost "hostname"
        ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
-p 2201 deployer@localhost
        ssh -i docker/vm_key -p 2201 root@localhost
```

3. Domain resolution + HTTPS with HTTP to HTTPS redirect

```
        ping -c 2 public.vm1.local
        curl -Ik http://public.vm1.local:8080
        curl -Ik https://public.vm1.local:8443
```

4. Ansible connectivity

```
        cd ansible
        ansible all -m ping
```

5. Playbook run twice, idempotency proof

```
        ansible-playbook site.yml
        ansible-playbook site.yml
```

6. Nginx dynamic pages on all VMs

```
        docker exec vm2 curl -s http://localhost
        docker exec vm3 curl -s http://localhost
```

7. Fixed-path routing + load balancing

```
        cd ..
        curl -k https://public.vm1.local:8443/vm2/
        curl -k https://public.vm1.local:8443/vm3/
        curl -k https://public.vm1.local:8443/app/
        curl -k https://public.vm1.local:8443/app/
```

8. Live failover test

```
        ( while true; do curl -sk -o /dev/null -w "%{http_code}\n"
https://public.vm1.local:8443/app/; sleep 1; done ) &
        docker stop vm2
```

```
        # wait ~8 seconds, narrate: "still 200s via vm3"
        docker start vm2
        # wait ~8 seconds, narrate: "vm2 rejoining, no manual reload"
        kill %1
        for i in 1 2 3 4 5 6; do curl -sk https://public.vm1.local:8443/app/
| grep Host; done
```

End of report.